LANGUAGE AGNOSTIC

CHEAT SHEETS

A Complete Reference for Bitwise Operations

Bits & Binary · AND · OR · XOR · NOT · Shifts · Masks & Flags · Bit Tricks · Bit Packing · Subset Enumeration · Popcount & Parity · Hamming Distance · Bloom Filters · XOR Cipher · Bitmask DP

8 REFERENCE SHEETS

CONTENTS

Covers AND · OR · XOR · NOT · Shifts · Masks · Packing · Subsets · Popcount (2025)

Begin → Sheet 01

Table of Contents

8 reference sheets · syntax · examples · notes

Bits & Binary

number bases · binary counting · bytes · hex · powers of two

WHAT IS A BIT?

A **bit** (binary digit) is the smallest unit of information in a computer — it can only be `0` or `1`, representing off or on, false or true, low voltage or high voltage. Everything a computer stores or processes — numbers, text, images, code — is ultimately a sequence of bits. Eight bits form a **byte**, which can represent 256 distinct values (0–255). Because each bit is a power of two, **binary (base-2)** is the natural number system for hardware: doubling the bit width squares the number of values you can represent.

KEY TERMS

- `bit` — single binary digit, 0 or 1
- `nibble` — 4 bits; exactly one hex digit
- `byte` — 8 bits; values 0–255
- `word` — machine-native size (16 / 32 / 64 bit)
- **LSB** — Least Significant Bit (rightmost, position 0)
- **MSB** — Most Significant Bit (leftmost, highest position)
- Unsigned integers: all bits are magnitude (0 → 2^n-1)
- Signed integers: MSB is the sign bit (two's complement)

DECIMAL · BINARY · HEX (0–15)

DEC	BINARY	HEX	DEC	BINARY	HEX
0	0000	0x0	8	1000	0x8
1	0001	0x1	9	1001	0x9
2	0010	0x2	10	1010	0xA
3	0011	0x3	11	1011	0xB
4	0100	0x4	12	1100	0xC
5	0101	0x5	13	1101	0xD
6	0110	0x6	14	1110	0xE
7	0111	0x7	15	1111	0xF

POWERS OF TWO

2^n	DECIMAL	HEX	COMMON USE
2^0	1	0x1	single flag bit
2^4	16	0x10	one nibble over
2^7	128	0x80	signed byte MSB
2^8	256	0x100	byte range (0–255)
2^{10}	1,024	0x400	1 KiB
2^{12}	4,096	0x1000	memory page size
2^{16}	65,536	0x10000	16-bit range / port max
2^{20}	1,048,576	0x100000	1 MiB
2^{31}	2,147,483,648	0x80000000	signed 32-bit MSB
2^{32}	4,294,967,296	0x100000000	unsigned 32-bit max+1

READING & WRITING BINARY / HEX LITERALS

```
// ── Binary literals (prefix 0b) ──────────────────────────
let a = 0b00001010;      // decimal 10
let b = 0b11110000;      // decimal 240

// ── Hex literals (prefix 0x) ─────────────────────────────
let c = 0xFF;            // decimal 255  (8 bits all set)
let d = 0x1A;            // decimal 26

// ── Octal literals (prefix 0o) ───────────────────────────
let e = 0o17;            // decimal 15

// ── Converting between bases ─────────────────────────────
(255).toString(2);       // "11111111"  — decimal → binary string
(255).toString(16);      // "ff"        — decimal → hex string
parseInt("1010", 2);     // 10          — binary string → decimal
parseInt("ff", 16);      // 255         — hex string → decimal

// ── Binary place values (reading 0b1011) ─────────────────
// bit 3  bit 2  bit 1  bit 0
//   1      0      1      1
//  (8) +  (0) +  (2) +  (1)  =  11
```

NOTES

- `0b` prefix — binary literal; supported in most modern languages
- `0x` prefix — hexadecimal literal; 2 hex digits = 1 byte
- `0o` prefix — octal (base-8); rarely needed today but appears in Unix permissions
- `toString(2)` — convert to binary string for inspection; language-specific
- `parseInt(str, base)` — parse a string in any base; second arg is the radix
- Place values double right-to-left: bit 0 = 1, bit 1 = 2, bit 2 = 4, bit 3 = 8…
- One hex digit always represents exactly 4 bits — use hex to read binary quickly

COMMON MISTAKES

- Counting bit positions from 1 — bit positions always start at `0` (rightmost = bit 0)
- Assuming integer width — always verify whether your language uses 32 or 64-bit integers before bitwise ops
- Confusing `0b1000` (8) with "the first bit is set" — bit 3 is set, not bit 0
- Ignoring sign extension — right-shifting a negative signed integer fills with 1s, not 0s

TIPS

- Memorize the 4-bit patterns for 0–15; a hex digit IS a nibble — reading `0xAB` instantly gives you `1010 1011`
- When debugging, print values in both hex and binary so you can see the bit layout clearly
- Powers of 2 that end in many zeros in binary always end in a round hex number — use this to sanity-check your math

SUMMARY

- `bit` — 0 or 1; `byte` — 8 bits; `nibble` — 4 bits = 1 hex digit
- Binary place values: bit N has value 2^N
- Hex is shorthand for binary — 2 hex digits = 1 byte, always
- Bit positions are zero-indexed from the right (LSB = bit 0)

The Six Operators

AND · OR · XOR · NOT · left shift · right shift · truth tables

HOW BITWISE OPERATORS WORK

Bitwise operators act **independently on every bit** in an integer — they do not carry or borrow between positions the way arithmetic does. Think of them as applying a logic gate to each column of bits simultaneously. **AND**, **OR**, **XOR**, and **NOT** are the logical operators; **left shift** and **right shift** move bits to different positions. All six operate in a single CPU instruction — they are the fastest operations available. The key insight is that *each bit position is completely independent*: what happens at bit 3 has no effect on bit 4.

PLAIN-ENGLISH RULES

- `AND (&)` — result is 1 only if *both* bits are 1
- `OR (|)` — result is 1 if *either* bit is 1
- `XOR (^)` — result is 1 if bits are *different*; 0 if same
- `NOT (~)` — flips every bit; in signed ints: `~x = -(x+1)`
- `<< n` — shift bits left by n; fills with 0s on right; equals $\times 2^n$
- `>> n` — shift bits right by n; unsigned fills 0s; signed may fill sign bit
- Precedence is low — always parenthesize: `(a & b) == c` not `a & b == c`
- XOR is its own inverse: `(a ^ b) ^ b == a` always

TRUTH TABLES — AND · OR · XOR · NOT

A	B	A & B	A | B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

SHIFT OPERATORS

EXPRESSION	BINARY	DECIMAL	NOTE
`1 << 0`	00000001	1	no shift
`1 << 1`	00000010	2	×2
`1 << 3`	00001000	8	×8
`1 << 7`	10000000	128	×128
`240 >> 1`	01111000	120	÷2
`240 >> 4`	00001111	15	÷16
`-8 >> 1`	11111100	-4	arithmetic (signed)

NOTES

- → `AND` — acts as a filter; bits not in the mask are zeroed out
- → `OR` — acts as a merger; any bit set in either operand is set in result
- → `XOR` — acts as a difference detector; identical bits cancel to 0
- → `~x = -(x+1)` — because NOT flips all bits and two's complement adds 1 to negate; `~0 = -1` always
- → `<< n` — same as multiplying by 2^n; bits shifted off the left edge are lost
- → `>> n` — arithmetic shift (signed) fills vacated bits with the sign bit; logical shift (unsigned) fills with 0
- → Precedence — bitwise operators have lower precedence than `==`; always use parentheses

ALL SIX OPERATORS IN ACTION (8-BIT EXAMPLES)

```
// Values used throughout
let a = 0b10110100;   // 180
let b = 0b11001010;   // 202

// ── AND (&) — keep bits where BOTH are 1 ──────────────
a & b;   // 0b10000000  =  128   (only bit 7 was 1 in both)

// ── OR (|) — keep bits where EITHER is 1 ──────────────
a | b;   // 0b11111110  =  254

// ── XOR (^) — keep bits where they DIFFER ─────────────
a ^ b;   // 0b01111110  =  126

// ── NOT (~) — flip every bit ──────────────────────────
~a;
// 0b01001011  =  -181  ← signed: ~x always equals -(x+1)
~0;
//                -1  ← all bits set in two's complement

// ── LEFT SHIFT (<<) — multiply by 2 per step ──────────
5 << 1;  // 10    (5 × 2)
5 << 3;  // 40    (5 × 8)

// ── RIGHT SHIFT (>>) — divide by 2 per step ───────────
40 >> 1; // 20    (40 ÷ 2)
40 >> 3; //  5    (40 ÷ 8)

// ── PRECEDENCE TRAP ───────────────────────────────────
a & b == 128;    // WRONG: parsed as a & (b == 128)
(a & b) == 128;  // correct
```

COMMON MISTAKES

- Using `~x` expecting a simple complement without sign — in signed integers `~5 = -6`, not `-5`
- Forgetting parentheses — `a & b == c` is not `(a & b) == c`; bitwise precedence is lower than comparison
- Mixing `&` (bitwise) with `&&` (logical) — they are entirely different operations
- Shifting by more than the integer width — behavior is undefined in C/C++; results vary by language

TIPS

- Remember the plain-English rules: AND = "both", OR = "either", XOR = "different", NOT = "flip all"
- XOR's self-inverse property (`(a^b)^b == a`) is used in encryption, swap tricks, and error correction
- Use unsigned integer types when you want `>>` to always fill with zeros (logical shift)

SUMMARY

- `&` AND — both 1 → 1; clears bits not in mask
- `|` OR — either 1 → 1; sets bits from mask
- `^` XOR — different → 1; toggles bits; self-inverse
- `~` NOT — flips all; `~x = -(x+1)` for signed integers
- `<<` / `>>` — multiply / divide by powers of two

Masks & Flags

WHAT IS A MASK?

A **mask** is just an integer used as a template to isolate specific bits in another integer. By combining a mask with AND, OR, XOR, or NOT you can **read, set, clear, or toggle individual bits** without disturbing any other bit in the value. This lets a single integer store many independent boolean flags — one per bit. This pattern is everywhere: Unix file permissions, network protocol headers, game entity state, hardware registers, and feature toggles all use it. The mask itself is simply `1 << N` to target bit N, or a combination of shifts and ORs for multiple bits at once.

BUILDING MASKS

- Mask for bit N: `1 << N`
- Mask for bits 0 and 3: `(1 << 0) | (1 << 3)`
- Mask for bits 0–3 (low nibble): `0xF` or `0b00001111`
- Mask for N contiguous bits starting at offset 0: `((1 << N) - 1) << 0`
- Inverted mask (for clearing): `~mask`
- Always name your masks as constants — never use bare numbers in flag operations
- Group related flags in the same byte/int and document reserved bits

THE FOUR FUNDAMENTAL MASK OPERATIONS

OPERATION	FORMULA	EFFECT
`Test a bit`	(value & mask) != 0	Returns truthy if bit is set
`Set a bit`	value | mask	Forces bit to 1, others unchanged
`Clear a bit`	value & ~mask	Forces bit to 0, others unchanged
`Toggle a bit`	value ^ mask	Flips bit, others unchanged
`Test all set`	(value & mask) == mask	All masked bits must be 1
`Test any set`	(value & mask) != 0	At least one masked bit is 1
`Clear all flags`	value = 0	Reset entire flags integer

WHY EACH FORMULA WORKS

OPERATION	REASON
`| mask`	OR with 1 always produces 1; OR with 0 preserves the original bit
`& ~mask`	NOT inverts the mask; AND with 0 forces bit to 0; AND with 1 preserves
`^ mask`	XOR with 1 flips; XOR with 0 preserves
`& mask != 0`	AND isolates the target bit(s); any non-zero result means at least one was set

NOTES

- → `1 << N` — builds a mask for bit N; bit 0 is rightmost
- → `|= FLAG` — compound assignment to set; leaves all other bits untouched
- → `& ~FLAG` — NOT inverts the mask (all 1s except target bit); AND then clears just that position
- → `^= FLAG` — toggle; apply twice to return to original state
- → `!== 0` — always compare to 0 when testing, not to the mask value (fails when flag is not at bit 0)
- → Compound test — OR the flags first to build a combined mask, then test with `== mask` for "all" or `!= 0` for "any"

PERMISSION FLAGS EXAMPLE

```
// ── Define flags as named constants ──────────
const READ    = 1 << 0;  // 0b00000001  =  1
const WRITE   = 1 << 1;  // 0b00000010  =  2
const EXECUTE = 1 << 2;  // 0b00000100  =  4
const ADMIN   = 1 << 7;  // 0b10000000  =  128

// ── Start with no permissions ────────────────
let perms = 0;           // 0b00000000

// ── SET: grant read and write ────────────────
perms |= READ;           // 0b00000001
perms |= WRITE;          // 0b00000011
perms |= EXECUTE;        // 0b00000111

// ── TEST: check a single permission ──────────
(perms & READ) !== 0;
       // true
(perms & ADMIN) !== 0;
       // false — admin not granted

// ── CLEAR: revoke execute ────────────────────
perms &= ~EXECUTE;   // 0b00000011

// ── TOGGLE: flip write permission ────────────
perms ^= WRITE;
       // 0b00000001 (write off)
perms ^= WRITE;
       // 0b00000011 (write on again)

// ── TEST MULTIPLE: must have both read AND write ─
(perms & (READ | WRITE)) === (READ | WRITE);
       // true
```

COMMON MISTAKES

- Testing with `== mask` instead of `!= 0` — only works when the flag is at bit 0; breaks for all others
- Clearing with `& mask` instead of `& ~mask` — AND with the mask keeps the bit, it doesn't clear it
- Using bare numbers like `perms & 4` instead of named constants — magic numbers make the code unreadable and error-prone
- Off-by-one in bit position — `1 << 8` is bit 8 (the 9th bit), not bit 7

TIPS

- Always define flag constants at the top of a module — if you ever need to add a flag, the bit assignment is in one place
- Comment which bits are reserved: `// bits 3-6: reserved, must be 0` — prevents future collision
- To print a readable flags report, loop from bit 0 to bit 31, test each, and print the flag name if set

SUMMARY

- Mask for bit N: `1 << N`
- Set: `value |= mask` · Clear: `value &= ~mask`
- Toggle: `value ^= mask` · Test: `(value & mask) != 0`
- Test all: `(value & mask) == mask` · Test any: `(value & mask) != 0`

Bit Tricks & Shortcuts

WHY BIT TRICKS EXIST

Bitwise operations execute in a single CPU clock cycle — faster than multiplication, division, or branching. The tricks on this sheet replace common operations with bit-level equivalents that compilers often cannot auto-discover. They appear in **competitive programming**, **embedded systems**, **graphics engines**, and **cryptography**.

Understanding them also deepens your intuition for how two's complement signed integers behave. Modern languages often provide built-in intrinsics (like `popcount`) that are even faster — always profile before assuming a manual trick beats a built-in.

TWO'S COMPLEMENT FACTS

- Negate: `-x == ~x + 1` (flip all bits, add 1)
- `~0 == -1` — all bits set equals negative one
- `~x == -(x+1)` — NOT is one less than negation
- Arithmetic right shift fills vacated bits with the sign bit
- `x >> 31` on a signed 32-bit int = 0 if positive, -1 if negative
- `x & -x` isolates the lowest set bit (uses negation trick)
- XOR swap: `a^=b; b^=a; a^=b` — fails if a and b are the same variable

NOTES

- → `x & 1` — the LSB is literally the ones column; it cannot lie about even/odd
- → `x & (x-1)` — subtracting 1 borrows through all the trailing zeros and flips the lowest 1; AND removes it
- → `x & -x` — two's complement negation flips all bits then adds 1, making every bit below the lowest 1 cancel; only the lowest 1 survives the AND
- → XOR swap works because XOR is associative and its own inverse — but *never* use it when a and b may be the same memory location (both become 0)
- → `x & (m-1)` — works only when m is a power of 2; the low bits of m-1 form an exact remainder mask
- → Branchless tricks matter most in hot loops where branch misprediction is costly; measure before optimizing

BIT TRICKS DEMONSTRATED

```
// ── Even / Odd ──────────────────────────────────────
7 & 1;            // 1  → odd
8 & 1;            // 0  → even

// ── Is power of two? ────────────────────────────────
let isPow2 = x => x > 0 && (x & (x - 1)) === 0;
isPow2(8);    // true   — 0b1000; 8-1=0b0111; AND = 0
isPow2(6);    // false  — 0b0110; 6-1=0b0101; AND = 0b0100 ≠ 0

// ── Clear lowest set bit (Kernighan's bit-count core) ──
let x = 0b10110100;   // 180
x & (x - 1);      // 0b10110000 — cleared the rightmost 1

// ── Isolate lowest set bit ──────────────────────────
x & (-x);         // 0b00000100 — only the rightmost 1 remains

// ── XOR Swap ────────────────────────────────────────
let a = 42, b = 17;
a ^= b;   // a = 59  (42 XOR 17)
b ^= a;   // b = 42  (17 XOR 59 = original a)
a ^= b;   // a = 17  (59 XOR 42 = original b)

// ── Mod by power of 2 ───────────────────────────────
100 % 16;        // 4  — standard modulo
100 & (16 - 1);  // 4  — same result, one instruction
```

CLASSIC BIT TRICKS REFERENCE

GOAL	FORMULA	HOW IT WORKS
`Multiply by 2ⁿ`	x << n	Each left shift doubles the value
`Divide by 2ⁿ (unsigned)`	x >> n	Each right shift halves (truncates toward zero)
`Even or odd?`	x & 1 == 0	LSB is 0 for even, 1 for odd — always
`Is power of two?`	x > 0 && (x & (x-1)) == 0	Powers of 2 have exactly one bit set; subtracting 1 flips all lower bits
`Clear lowest set bit`	x & (x - 1)	x-1 turns off the lowest 1 and turns on all bits below it; AND removes it
`Isolate lowest set bit`	x & (-x)	-x in two's complement flips bits up to and including lowest 1; AND keeps just that bit
`XOR swap (no temp)`	a^=b; b^=a; a^=b	XOR self-inverse cancels each value in turn; unsafe if a and b alias same address
`Branchless absolute value`	mask = x>>31; (x^mask)-mask	Arithmetic shift fills mask with sign bit (0 or -1); XOR+sub negates if negative
`Branchless min(a, b)`	b ^ ((a^b) & -(a<b))	Boolean comparison becomes 0 or -1; selects a or b without branch
`Round down to power of 2`	1 << floor(log2(x))	Or use CLZ: `1 << (31 - clz(x))`
`Mod by power of 2`	x & (m - 1)	Only works when m is a power of 2; replaces expensive modulo

COMMON MISTAKES

- XOR swap with aliased pointers — if `&a == &b`, `a ^= b` sets `a` to 0 and you lose both values
- Using `x & (m-1)` for modulo when `m` is not a power of 2 — gives wrong results silently
- Arithmetic right shift on unsigned types in C — implementation-defined; cast to signed or use language-specific unsigned right shift

TIPS

- `x & (x-1)` in a loop until `x == 0` counts set bits in O(k) where k is the number of 1s — Kernighan's algorithm
- Most languages now have a built-in popcount / bit_count — prefer it over manual loops; it compiles to a single hardware instruction
- Profile before optimizing with bit tricks — modern compilers often generate the same code from readable arithmetic

SUMMARY

- `x & 1` — parity (0=even, 1=odd)
- `x & (x-1) == 0` — power-of-two check
- `x & (x-1)` — clear lowest set bit
- `x & -x` — isolate lowest set bit
- `x & (m-1)` — fast modulo when m is power of 2

Packing & Fields

STORING MULTIPLE VALUES IN ONE INTEGER

When memory or bandwidth is constrained, multiple small integers can be **packed** into a single larger integer by assigning each value a specific range of bits — called a **bit field**. Packing uses left-shift and OR to place values; **unpacking** uses right-shift and AND-mask to extract them. This technique is ubiquitous: ARGB color values (4 × 8-bit channels in one 32-bit int), IP and TCP headers, UTF-8 byte structure, audio sample formats, and game entity data all rely on bit packing. The discipline is to define field *offset* (which bit position a field starts at) and *width* (how many bits it occupies), then derive the mask automatically as `((1 << width) - 1) << offset`.

ARGB 32-BIT COLOR LAYOUT

CHANNEL	BITS	OFFSET	MASK
Alpha	31 – 24	24	0xFF000000
Red	23 – 16	16	0x00FF0000
Green	15 – 8	8	0x0000FF00
Blue	7 – 0	0	0x000000FF

GENERIC PACK / UNPACK / REPLACE

OPERATION	FORMULA
Build mask	((1 << width) - 1)
Pack one field	(value & mask) << offset
Pack two fields	(a << offA) | (b << offB)
Unpack a field	(packed >> offset) & mask
Clear a field	packed & ~(mask << offset)
Replace a field	(packed & ~(mask << off)) | ((newVal & mask) << off)

NOTES

- → `& CH_MASK` before shift — clamps input to 8 bits; prevents dirty high bits from bleeding into adjacent fields
- → Encode uses `|` to merge all four shifted channels — OR is safe because no two fields overlap
- → Decode: shift the field down to bit 0, then AND with the channel mask to discard anything above it
- → Replace = clear field with `& ~(mask << offset)` to zero that region, then OR in the new shifted value
- → Result `0xFF6495ED` — read as: FF=alpha, 64=red (hex 100), 95=green (hex 149), ED=blue (hex 237)
- → This same pattern applies to any packed format: IP headers, audio samples, UTF-8 continuation bytes, etc.

FIELD VOCABULARY

- **offset** — bit position where the field starts (bit 0 = rightmost)
- **width** — number of bits in the field
- **mask** — `(1 << width) - 1` — a run of *width* 1-bits
- Pack: `(value & mask) << offset` then OR into target
- Unpack: `(packed >> offset) & mask`
- Replace: clear field first with `& ~(mask << offset)`, then OR in new value
- Always mask input before packing to prevent dirty high bits corrupting adjacent fields

ENCODE & DECODE AN ARGB COLOR

```
// ── Channel constants ──────────────────────────────────
const ALPHA_OFFSET = 24, RED_OFFSET = 16,
      GREEN_OFFSET = 8,  BLUE_OFFSET = 0;
const CH_MASK = 0xFF;   // 8-bit mask for any single channel

// ── Encode: pack four 8-bit values into one 32-bit int ──
function encodeARGB(a, r, g, b) {
  return ((a & CH_MASK) << ALPHA_OFFSET) |
         ((r & CH_MASK) << RED_OFFSET)   |
         ((g & CH_MASK) << GREEN_OFFSET) |
         ((b & CH_MASK) << BLUE_OFFSET);
}

let color = encodeARGB(255, 100, 149, 237);
// color = 0xFF6495ED  (cornflower blue, fully opaque)

// ── Decode: unpack each channel ────────────────────────
let alpha = (color >> ALPHA_OFFSET) & CH_MASK;  // 255
let red   = (color >> RED_OFFSET)   & CH_MASK;  // 100
let green = (color >> GREEN_OFFSET) & CH_MASK;  // 149
let blue  = (color >> BLUE_OFFSET)  & CH_MASK;  // 237

// ── Replace: update alpha without touching RGB ─────────
function setAlpha(color, newAlpha) {
  return (color & ~(CH_MASK << ALPHA_OFFSET)) |
         ((newAlpha & CH_MASK) << ALPHA_OFFSET);
}
let semitransparent = setAlpha(color, 128);
// red/green/blue unchanged; alpha is now 128
```

COMMON MISTAKES

- Forgetting to mask input before packing — if red = 300 (0x12C), the high bits spill into the alpha field silently
- Endianness confusion — packed ints serialize to bytes differently on big-endian vs little-endian machines; specify byte order explicitly in network/file formats
- Assuming 32-bit int — JavaScript bitwise ops are 32-bit signed; packing more than 31 bits of magnitude requires BigInt or two separate ints

TIPS

- Define OFFSET and WIDTH constants together — never one without the other; derive MASK from WIDTH so they stay in sync
- Draw a bit-field diagram in a comment above any packing struct: `// [31..24]=A [23..16]=R [15..8]=G [7..0]=B`
- Validate inputs fit their field before packing: `assert(value >= 0 && value <= 255)` — silent overflow is hard to debug

SUMMARY

- Pack with: `(value & mask) << offset | ...`
- Unpack with: `(packed >> offset) & mask`
- Replace: clear field (`& ~(mask << off)`), then OR in new value
- Always mask inputs before packing; always AND after unpacking

Subset Enumeration

A BIT PER ELEMENT — REPRESENTING SETS

If a collection has **N elements**, any subset can be represented by an N-bit integer where **bit i is 1 if element i is in the subset**, and 0 if it is not. Counting from `0` to $2^N - 1$ visits every possible subset exactly once — all 2^N of them. This is **bitmask enumeration**: the foundation of bitmask dynamic programming (bitmask DP), which solves NP-hard problems like the Travelling Salesman Problem, minimum cost scheduling, and set-cover on small N (typically N ≤ 20). The key insight is that set union is OR, intersection is AND, complement is XOR-with-full-mask, and membership test is a single bit check — all in one CPU instruction.

COMPLEXITY NOTES

- All subsets of N elements: **2^N** iterations
- All subsets of all subsets: **3^N** total (sum of $C(N,k) \times 2^k$)
- N=20 → ~1M subsets; N=25 → ~33M; N=30 → ~1B (slow)
- Bitmask DP state space: often $O(2^N \times N)$ time, $O(2^N)$ space
- Gosper's Hack iterates only subsets of a given mask in O(subsets) — no wasted iterations
- Empty set (mask=0) is always a valid subset — don't skip it
- Full set of N elements: `(1 << N) - 1`

NOTES

- → Counting mask from `0` to `2^N-1` covers every subset because binary counting visits every combination of N bits
- → `(mask >> i) & 1` — shift element i's bit down to position 0, then AND to isolate it
- → Gosper's step `(s-1) & mask` — subtracting 1 borrows through trailing zeros and flips the lowest set bit; AND with mask keeps the result within the parent set's bits
- → Gosper's loop exits at `s=0` (empty set); the loop condition `s > 0` means the empty set is not visited — add it manually if needed
- → Total iterations for all subsets of all subsets of an N-element set: 3^N (each element is in the outer set, inner set, or neither)
- → Bitmask DP: store results indexed by subset mask in a `dp[mask]` array — transition by adding or removing one element at a time

ENUMERATE ALL SUBSETS + GOSPER'S HACK

```
const items = ["apple", "banana", "cherry"];
const N = items.length;   // 3 elements → 8 subsets (2³)

// ── Enumerate ALL subsets of items ──────────────────────
for (let mask = 0; mask < (1 << N); mask++) {
  let subset = [];
  for (let i = 0; i < N; i++) {
    if ((mask >> i) & 1) subset.push(items[i]);
  }
  console.log(`mask=${mask.toString(2).padStart(N,'0')}`, subset);
}
// mask=000 []
// mask=001 ["apple"]
// mask=010 ["banana"]
// mask=011 ["apple","banana"]
// mask=100 ["cherry"]
// mask=101 ["apple","cherry"]
// mask=110 ["banana","cherry"]
// mask=111 ["apple","banana","cherry"]

// ── Enumerate only subsets of a given mask (Gosper's Hack) ──
// E.g. find all subsets of the mask that includes apple+cherry
let target = 0b101;   // bits 0 and 2 set = {apple, cherry}

for (let s = target; s > 0; s = (s - 1) & target) {
  // visits: 0b101, 0b100, 0b001
  // i.e. {apple,cherry}, {cherry}, {apple}
  // empty set (s=0) exits loop — handle separately if needed
}
```

SUBSET OPERATIONS REFERENCE

OPERATION	FORMULA	MEANING
Is element i in S?	(S >> i) & 1	Test bit i of S
Add element i to S	S | (1 << i)	Set bit i
Remove element i from S	S & ~(1 << i)	Clear bit i
Toggle element i	S ^ (1 << i)	XOR bit i
Is S a subset of T?	(S & T) == S	All bits of S also in T
Union of S and T	S | T	Elements in either set
Intersection of S and T	S & T	Elements in both sets
Difference S - T	S & ~T	Elements in S but not T
Complement of S (N-elem)	S ^ ((1 << N) - 1)	Flip all N bits
Size of S (# elements)	popcount(S)	Count set bits
Full set (all N elements)	(1 << N) - 1	All N bits set to 1
Next subset of mask	(S - 1) & mask	Gosper's sub-mask step

COMMON MISTAKES

- Using N > 30 with 32-bit integers — 2^{31} = 2 billion iterations; use 64-bit integers for N up to 62, or a different algorithm above that
- Off-by-one: full set is `(1 << N) - 1`, not `(1 << N)` — the latter is one past the end and overflows for N=32
- Skipping the empty set (mask=0) — it is a valid subset and often a base case in bitmask DP

TIPS

- Use `popcount(mask)` inside the loop to know how many elements are in the current subset — useful for filtering by size
- Bitmask DP is the standard approach for TSP, minimum Steiner tree, and scheduling problems when N ≤ 20 — learn the pattern once, apply everywhere
- Draw out the 3-element case by hand (8 rows, 3 columns) before coding a bitmask DP — it immediately reveals the transition structure

SUMMARY

- Bit i = whether element i is in the subset
- Loop `0` → `(1<<N)-1` to enumerate all 2^N subsets
- Gosper's `(s-1)&mask` iterates only subsets of a given mask
- Union=OR, Intersection=AND, Difference=`&~`, Complement=XOR full-mask

Sheet 07

Bit Counting & Parity

popcount · CLZ · CTZ · Hamming distance · parity · Kernighan's algorithm

TREATING BITS AS DATA

Beyond using bits as flags or packed fields, many algorithms treat the **bit pattern of a number as the data to analyze**. **Population count** (popcount) counts how many bits are 1. **Parity** checks whether that count is even or odd — used in serial communication, RAID, and error-correcting codes. **Hamming distance** counts positions where two bit patterns differ, and is computed in a single expression as `popcount(a ^ b)`. **CLZ/CTZ** (count leading/trailing zeros) locate the highest and lowest set bits efficiently and are used in fast logarithm computation, memory allocators, and scheduler priority queues. All of these have hardware intrinsics in modern CPUs — always prefer the built-in over a manual loop.

QUICK FACTS

- `popcount(x)` — number of 1-bits; also called "Hamming weight"
- `popcount(x) & 1` — parity (0=even number of 1s, 1=odd)
- `popcount(a ^ b)` — Hamming distance between a and b
- `clz(x)` — count leading zeros (undefined for x=0 in most languages)
- `ctz(x)` — count trailing zeros; equals position of lowest set bit
- Highest set bit position: `31 - clz(x)` (for 32-bit int)
- Kernighan's popcount: loop `x &= (x-1)` until 0; runs in O(set bits)
- Built-in intrinsics compile to a single hardware instruction on x86/ARM

BIT-COUNTING OPERATIONS

OPERATION	NAIVE APPROACH	BUILT-IN / FAST
Popcount	Loop & shift each bit	popcount() / bit_count()
Count leading zeros	Loop from MSB inward	clz() / __builtin_clz()
Count trailing zeros	Loop from LSB inward	ctz() / __builtin_ctz()
Highest set bit	floor(log2(x))	31 - clz(x)
Parity	popcount(x) % 2	popcount(x) & 1
Hamming distance	XOR then count bits	popcount(a ^ b)
Bit reversal	Loop and rebuild	Language-specific intrinsic

KERNIGHAN'S ALGORITHM TRACED

STEP	X (BINARY)	X&(X-1)	COUNT
start	10110100	—	0
iter 1	10110100	10110000	1
iter 2	10110000	10100000	2
iter 3	10100000	10000000	3
iter 4	10000000	00000000	4
done	00000000	x=0, exit	4 ✓

PARITY, HAMMING DISTANCE & KERNIGHAN POPCOUNT

```
// ── Kernighan's popcount ──────────────────────────────
function popcount(x) {
  let count = 0;
  while (x !== 0) { x &= (x - 1); count++; }
  return count;
}
popcount(0b10110100);   // 4  (four 1-bits)

// ── Parity (even=0, odd=1 number of set bits) ─────────
function parity(x) { return popcount(x) & 1; }
parity(0b10110100);     // 0  — even number of 1s
parity(0b10110101);     // 1  — odd number of 1s

// ── Hamming distance: how many bit positions differ? ──
function hammingDist(a, b) { return popcount(a ^ b); }
hammingDist(0b1100, 0b1010);   // 2  — bits 1 and 2 differ

// ── Hamming distance use-case: error detection ────────
// Two codewords with Hamming distance ≥ 2 can detect 1-bit errors
// Two codewords with Hamming distance ≥ 3 can correct 1-bit errors

// ── Count trailing zeros = position of lowest set bit ─
function ctz(x) {
  if (x === 0) return 32;
  return popcount((x & -x) - 1);   // isolate lowest bit, count belo
}
ctz(0b10110100);   // 2  — lowest set bit is at position 2

// ── Gosper's Hack: next int with same popcount ────────
function nextSameBitCount(x) {
  let lo = x & -x;                  // isolate lowest 1
  let hi = x + lo;                  // carry that 1 into next positi
  return hi | (((x ^ hi) / lo) >>> 2); // restore lower bits
}
nextSameBitCount(0b0011);   // 0b0101  (both have 2 set bits)
nextSameBitCount(0b0101);   // 0b0110
```

NOTES

- → `x &= (x-1)` — Kernighan's step; clears the lowest set bit each iteration; loop runs exactly popcount(x) times
- → `popcount & 1` — parity in one operation; used in serial comms to append a check bit so total 1s are always even or always odd
- → `a ^ b` — XOR produces a 1 at every position where a and b differ; popcount of that is the Hamming distance
- → Hamming distance is fundamental to error-correcting codes (ECC RAM, QR codes, RAID) and nearest-neighbor search
- → `x & -x` — isolates lowest set bit; CTZ = popcount of all the bits below it = popcount((x&-x)-1)
- → Gosper's Hack — generates all N-bit integers with exactly K bits set in ascending order; useful in combinatorics and bitmask DP enumeration

COMMON MISTAKES

- Calling `clz(0)` — result is undefined in C/C++ and many languages; always guard with `if (x != 0)`
- Implementing popcount as a bit-shift loop when the language has a built-in — the intrinsic is a single hardware instruction, the loop is 32
- Confusing Hamming weight (popcount of one value) with Hamming distance (popcount of XOR of two values)

TIPS

- Always use the language's built-in popcount — Python `bin(x).count('1')` or `x.bit_count()`, Java `Integer.bitCount()`, C++ `__builtin_popcount()`
- Parity byte in serial comms: XOR all data bytes together — the result's parity bit is your check byte
- Gosper's Hack is the canonical way to enumerate all size-K subsets of N elements in bitmask DP problems

SUMMARY

- `popcount(x)` — count of 1-bits; use built-in intrinsic
- `popcount(x) & 1` — parity (0=even, 1=odd)
- `popcount(a ^ b)` — Hamming distance between a and b
- `x &= (x-1)` — Kernighan's loop; clears lowest set bit each step

Real-World Patterns

WHERE BITWISE LIVES IN THE WILD

Bitwise operations appear across almost every domain of software. **Unix permissions** use a 9-bit field (rwxrwxrwx). **Network protocol headers** (IPv4, TCP, UDP) are dense bit-packed structures parsed with shift-and-mask. **Graphics** packs ARGB channels into 32-bit ints for GPU upload. **Game engines** store entity state (alive, visible, collidable) in a flags byte to update thousands of objects in tight loops. **Bloom filters** use OR to insert and AND to query a compact probabilistic set. **XOR ciphers** encrypt by XORing with a key and decrypt the same way (XOR is self-inverse). Understanding these patterns lets you read and write low-level code fluently across any language.

DOMAIN → BITWISE PATTERN

- **Unix permissions** — rwx × 3 groups, 9 bits; AND to test, OR to grant, `&~` to revoke
- **IPv4 / TCP headers** — multi-width fields; shift + mask to parse each field
- **ARGB color** — 4 × 8-bit channels in 32-bit int; shift + OR to encode
- **Game entity flags** — alive/visible/collidable bits; toggle with XOR in bulk update loops
- **Bloom filter** — OR to insert bit positions; AND to query membership
- **XOR cipher** — XOR each byte with key byte; XOR again to decrypt
- **Bitmask DP** — subset mask as dp table index; enumerate with Gosper's Hack

DOMAIN REFERENCE MAP

DOMAIN	WHAT'S PACKED	KEY OPERATION
Unix permissions	read/write/execute × owner/group/other	AND to test; OR grant; &~ revoke
IPv4 header	version, IHL, DSCP, TTL, flags, offset	shift + mask to parse each field
TCP flags byte	SYN, ACK, FIN, RST, PSH, URG (6 bits)	AND to test individual control flags
ARGB color	4 × 8-bit channels in 32-bit int	shift + OR encode; shift + AND decode
Game entity state	alive, visible, collidable, animated…	OR add; &~ remove; XOR toggle
Bloom filter	N hash → N bit positions in one int	OR insert; AND query all bits
XOR cipher	plaintext byte ⊕ key byte	XOR encrypts and decrypts identically
UTF-8 encoding	continuation bytes: 10xxxxxx pattern	AND 0xC0 == 0x80 to detect cont. byte

REUSABLE CODE PATTERNS

PATTERN	PSEUDOCODE
Define flags	FLAG_A = 1 << 0; FLAG_B = 1 << 1
Set flag	state |= FLAG_A
Clear flag	state &= ~FLAG_A
Toggle flag	state ^= FLAG_A
Test flag	(state & FLAG_A) != 0
Test all set	(state & (A|B)) == (A|B)
Test any set	(state & (A|B)) != 0
Extract N-bit field	(value >> offset) & ((1 << N) - 1)
Insert N-bit field	(value & ~(mask << off)) | ((field & mask) << off)

MINI BLOOM FILTER & XOR CIPHER

```
// == BLOOM FILTER (32-bit, 3 hash functions) ==========
let filter = 0;

// Simple hash functions — map a string to a bit position 0–31
const h1 = s => s.charCodeAt(0) % 32;
const h2 = s => (s.length * 7) % 32;
const h3 = s => (s.charCodeAt(0) * 31 + s.length) % 32;

function bloomInsert(word) {
  filter |= (1 << h1(word));  // set bit for each hash
  filter |= (1 << h2(word));
  filter |= (1 << h3(word));
}

function bloomQuery(word) {
  let bits = (1 << h1(word)) | (1 << h2(word)) | (1 << h3(word));
  return (filter & bits) === bits;  // all bits must be set
}

bloomInsert("cat");
bloomInsert("dog");
bloomQuery("cat");   // true  — definitely inserted
bloomQuery("rat");   // false — definitely NOT inserted
bloomQuery("bat");   // true  — FALSE POSITIVE (possible!)

// == XOR CIPHER ========================================
function xorCipher(data, key) {
  // Works for both encrypt and decrypt (XOR is self-inverse)
  return data.map((byte, i) => byte ^ key[i % key.length]);
}

let plaintext  = [72, 101, 108, 108, 111]; // "Hello" as bytes
let key        = [0xAB, 0xCD];
let ciphertext = xorCipher(plaintext, key); // encrypted
let recovered  = xorCipher(ciphertext, key);
            // decrypted → [72,101,108,108,111]
```

NOTES

- Bloom filter insert — OR in bit positions; never clears bits, so items cannot be removed (use Counting Bloom Filter for deletion)
- Bloom filter query — AND all expected bits; if any are 0, the item was definitely never inserted; if all are 1, it probably was (false positives possible)
- False positives — occur when hash collisions set all an item's bits even though it was never inserted; tunable with more bits and more hash functions
- XOR cipher — `plain ^ key = cipher`; `cipher ^ key = plain`; XOR is its own inverse, so encrypt and decrypt are identical functions
- XOR key reuse — reusing the same key for two plaintexts lets an attacker recover the XOR of the plaintexts; use each key only once (one-time pad)
- Real crypto — XOR is a building block inside AES, SHA, and ChaCha20; never use raw XOR as your sole encryption layer

COMMON MISTAKES

- Using `&` (bitwise) where you mean `&&` (logical) — `1 & 2` is 0, but `1 && 2` is truthy; they are completely different
- Reusing a XOR cipher key — two ciphertexts with the same key XOR to reveal the XOR of the plaintexts (the "two-time pad" attack)
- Forgetting to mask after extracting a signed field — sign extension can make a positive field value appear negative
- Adding a new flag in the wrong bit position — corrupts serialized data that other code already depends on; always append new flags at the highest unused bit

TIPS

- When reading protocol specs or binary file format docs, immediately draw a bit-field diagram and write named mask constants before any other code
- Version your flag layouts — if you serialize flags to disk or network, document the version and never change existing bit assignments
- XOR is the universal "difference detector" — if `a ^ b == 0` then a and b are identical; use it to compare hashes, detect corruption, or find changed bits

SUMMARY

- Flags: OR set · `&~` clear · XOR toggle · `& != 0` test
- Field extract: `(val >> off) & mask` · insert: clear then OR
- Bloom filter: OR insert · AND-test query · false positives possible, never false negatives
- XOR is self-inverse: same function encrypts and decrypts · never reuse keys